

Supermart Sales Analysis

This notebook performs an exploratory data analysis on the Supermart sales dataset. It includes data preprocessing, visualization, and insights generation to help improve business strategy.

Importing the libraries

Data Visualization

Visualize the data to identify trends and patterns.

In [3]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline

import warnings
warnings.filterwarnings('ignore')
```

Load the dataset

Load Dataset

We begin by importing the dataset using `pandas` .

In [4]:

```
supermart = pd.read_csv(
    r"E:\Unified Mentor Project\Supermart Grocery Sales\Supermart Grocery Sales - Retail
")

supermart.head()
```

Out[4]:

	Order ID	Customer Name	Category	Sub Category	City	Order Date	Region	Sales	Discount	Profit
0	OD1	Harish	Oil & Masala	Masalas	Vellore	11-08-2017	North	1254	0.12	401.28
1	OD2	Sudha	Beverages	Health Drinks	Krishnagiri	11-08-2017	South	749	0.18	149.80
2	OD3	Hussain	Food Grains	Atta & Flour	Perambalur	06-12-2017	West	2360	0.21	165.20
3	OD4	Jackson	Fruits & Veggies	Fresh Vegetables	Dharmapuri	10-11-2016	South	896	0.25	89.60



In [5]:

Check the shape of the dataset
supermart.shape

Out[5]: (9994, 11)

Data Preprocessing

1. Check the data types of the columns, missing values and handling them

Descriptive Statistics

Summary of numerical features in the dataset.

In [6]:

supermart.select_dtypes(include=["float", "int"]).describe().round(2)

Out[6]:

	Sales	Discount	Profit
count	9994.00	9994.00	9994.00
mean	1496.60	0.23	374.94
std	577.56	0.07	239.93
min	500.00	0.10	25.25
25%	1000.00	0.16	180.02
50%	1498.00	0.23	320.78
75%	1994.75	0.29	525.63
max	2500.00	0.35	1120.95

Descriptive Statistics

Summary of numerical features in the dataset.

In [7]:

supermart.describe()

Out[7]:

	Sales	Discount	Profit
count	9994.000000	9994.000000	9994.000000
mean	1496.596158	0.226817	374.937082
std	577.559036	0.074636	239.932881
min	500.000000	0.100000	25.250000
25%	1000.000000	0.160000	180.022500

50%	1498.000000	0.230000	320.780000
75%	1994.750000	0.290000	525.627500
max	2500.000000	0.350000	1120.950000

```
In [8]: # Checking the data types
supermart.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Order ID        9994 non-null   object
1   Customer Name    9994 non-null   object
2   Category         9994 non-null   object
3   Sub Category     9994 non-null   object
4   City             9994 non-null   object
5   Order Date       9994 non-null   object
6   Region           9994 non-null   object
7   Sales            9994 non-null   int64
8   Discount         9994 non-null   float64
9   Profit           9994 non-null   float64
10  State            9994 non-null   object
dtypes: float64(2), int64(1), object(8)
memory usage: 859.0+ KB
```

Missing Values Check

Identify and count any missing values in the dataset.

```
In [9]: # checking the missing values
supermart.isnull().sum()
```

```
Out[9]: Order ID        0
Customer Name    0
Category         0
Sub Category     0
City             0
Order Date       0
Region           0
Sales            0
Discount         0
Profit           0
State            0
dtype: int64
```

```
In [10]: supermart.columns = supermart.columns.str.replace(" ", "_")
```

Aggregated Analysis

Group data by categories (e.g., Segment, Category) to analyze sales and profit.

```
In [11]: supermart.groupby(["Category", "Sub_Category"])["Sales"].sum().sort_values(
ascending=False
).head(10).rolling(1).sum()
```

```
Out[11]: Category      Sub_Category
Beverages      Health Drinks    1051439.0
```

```
Snacks      Soft Drinks      1033874.0
Bakery       Breads & Buns      742586.0
Snacks       Noodles            735435.0
              Chocolates      733898.0
Oil & Masala  Masalas           697480.0
Bakery       Cakes             685612.0
              Biscuits            684083.0
Oil & Masala  Spices            672876.0
Name: Sales, dtype: float64
```

Aggregated Analysis

Group data by categories (e.g., Segment, Category) to analyze sales and profit.

```
In [12]: supermart.groupby(["Category", "City", "Region"]).agg({"Sales": "sum"}).sort_values(
        by=["Sales"], ascending=False
        ).reset_index()
```

Out[12]:

	Category	City	Region	Sales
0	Bakery	Salem	East	45524
1	Oil & Masala	Pudukottai	East	44795
2	Fruits & Veggies	Bodi	West	44550
3	Eggs, Meat & Fish	Karur	West	44254
4	Food Grains	Bodi	West	43681
...	...	...	...	...
668	Fruits & Veggies	Tenkasi	South	6079
669	Beverages	Chennai	South	5496
670	Fruits & Veggies	Cumbum	South	5339
671	Eggs, Meat & Fish	Pudukottai	South	3275
672	Oil & Masala	Vellore	North	1254

673 rows × 4 columns

```
In [13]: ## 2. Converting Date columns to DateTime format
from dateutil.parser import parse
import pandas as pd

supermart["Order_Date"] = supermart["Order_Date"].apply(
    lambda x: parse(x) if isinstance(x, str) else x
)

supermart["Order_Date"].dtype

supermart["Order_Date"].head()
```

Out[13]: 0 2017-11-08
1 2017-11-08
2 2017-06-12
3 2016-10-11
4 2016-10-11
Name: Order_Date, dtype: datetime64[ns]

```
In [14]: # Extract day, month and year from the Order Date column

supermart["Order_Year"] = supermart["Order_Date"].dt.year
supermart["Order_Month"] = supermart["Order_Date"].dt.month
supermart["Order_Day"] = supermart["Order_Date"].dt.day

supermart["Order_Year"].head()
```

Out[14]: 0 2017
1 2017
2 2017
3 2016
4 2016
Name: Order_Year, dtype: int32

```
In [15]: supermart.head()
```

Out[15]:

	Order_ID	Customer_Name	Category	Sub_Category	City	Order_Date	Region	Sales
0	OD1	Harish	Oil & Masala	Masalas	Vellore	2017-11-08	North	1254
1	OD2	Sudha	Beverages	Health Drinks	Krishnagiri	2017-11-08	South	749
2	OD3	Hussain	Food Grains	Atta & Flour	Perambalur	2017-06-12	West	2360
3	OD4	Jackson	Fruits & Veggies	Fresh Vegetables	Dharmapuri	2016-10-11	South	896
4	OD5	Ridhesh	Food Grains	Organic Staples	Ooty	2016-10-11	South	2355

Descriptive Statistics

Summary of numerical features in the dataset.

```
In [16]: # Check the data types again
supermart.describe().T.round(2)
```

Out[16]:

	count	mean	min	25%	50%	75%	max	
Order_Date	9994	2017-04-30 05:17:08.056834048	2015-01-03 00:00:00	2016-05-23 00:00:00	2017-06-26 00:00:00	2018-05-14 00:00:00	2018-12-30 00:00:00	N
Sales	9994.0	1496.596158	500.0	1000.0	1498.0	1994.75	2500.0	577.5596
Discount	9994.0	0.226817	0.1	0.16	0.23	0.29	0.35	0.0746
Profit	9994.0	374.937082	25.25	180.0225	320.78	525.6275	1120.95	239.9326
Order_Year	9994.0	2016.722233	2015.0	2016.0	2017.0	2018.0	2018.0	1.1235
Order_Month	9994.0	7.809686	1.0	5.0	9.0	11.0	12.0	3.2846
Order_Day	9994.0	15.468381	1.0	8.0	15.0	23.0	31.0	8.7487

Exploratory Data Analysis

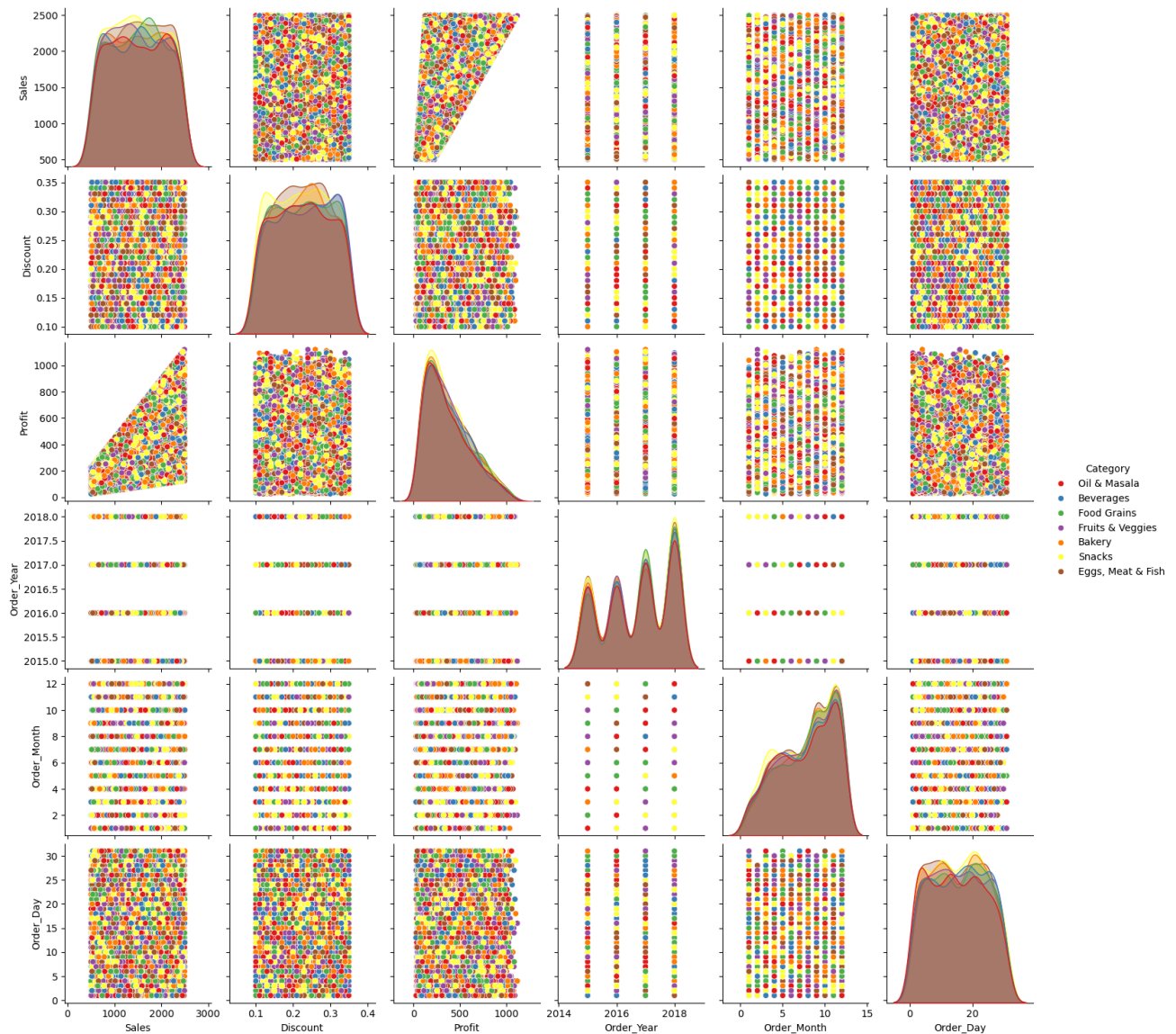
Data Visualization

Visualize the data to identify trends and patterns.

```
In [17]: plt.figure(figsize=(10, 6))

sns.pairplot(supermart, hue="Category", palette="Set1")
plt.show()
```

<Figure size 1000x600 with 0 Axes>



Aggregated Analysis

Group data by categories (e.g., Segment, Category) to analyze sales and profit.

```
In [ ]: # --- Analyze Correlation ---
print("Correlation Matrix (Sales, Discount, Profit):")
print(supermart[["Sales", "Discount", "Profit"]].corr())
```

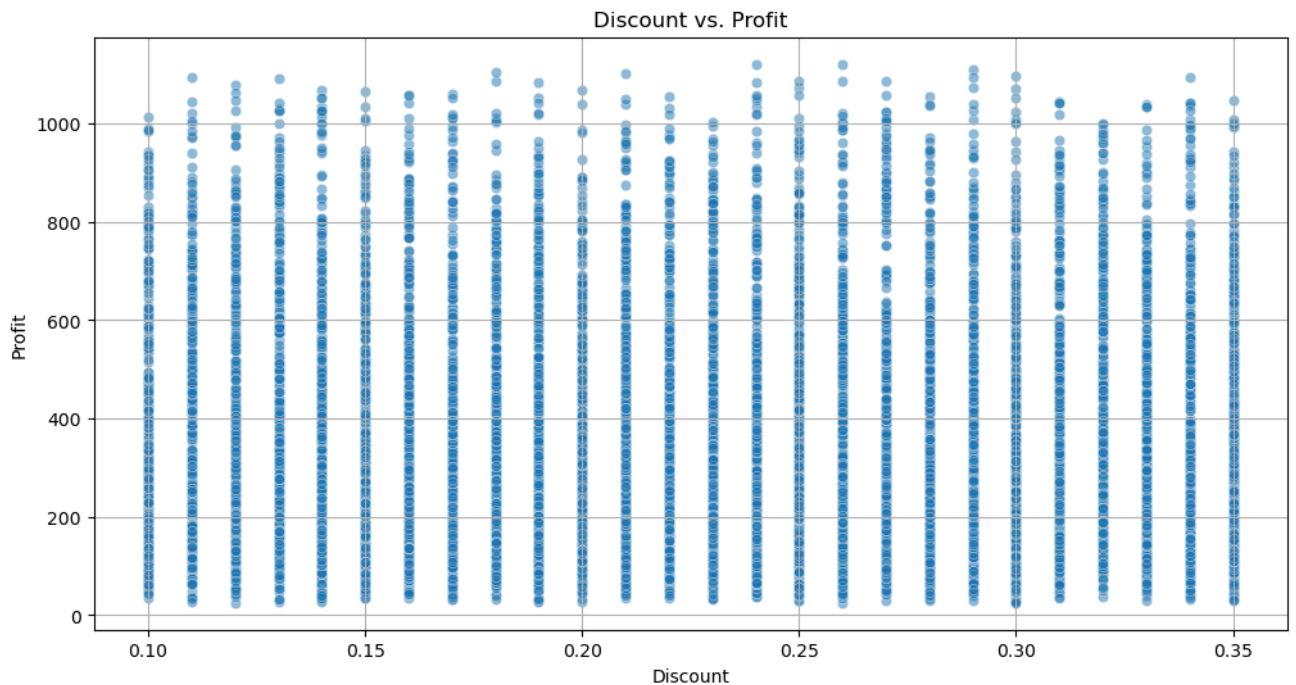
Correlation Matrix (Sales, Discount, Profit):

	Sales	Discount	Profit
Sales	1.000000	-0.005512	0.605349
Discount	-0.005512	1.000000	0.000017
Profit	0.605349	0.000017	1.000000

In []:

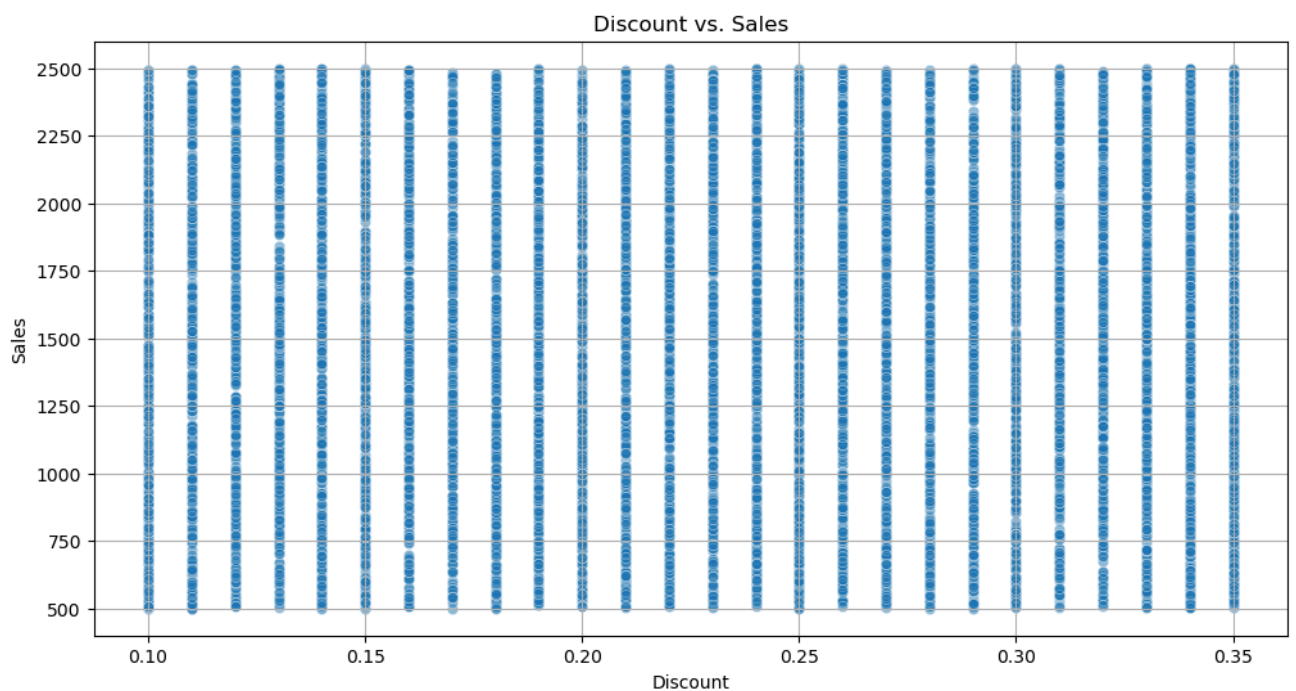
```
# --- Visualize Discount vs. Sales/Profit ---
```

```
# Scatter plot of Discount vs. Profit
plt.figure(figsize=(12, 6))
sns.scatterplot(data=supermart, x="Discount", y="Profit", alpha=0.5)
plt.title("Discount vs. Profit")
plt.xlabel("Discount")
plt.ylabel("Profit")
plt.grid(True)
plt.show()
```



In [35]:

```
# Scatter plot of Discount vs. Sales
plt.figure(figsize=(12, 6))
sns.scatterplot(data=supermart, x="Discount", y="Sales", alpha=0.5)
plt.title("Discount vs. Sales")
plt.xlabel("Discount")
plt.ylabel("Sales")
plt.grid(True)
plt.show()
```



In []:

```
# --- Analyze Profitability by Discount Bins ---
```

```

# Create discount bins (e.g., low, medium, high)
discount_bins = [0, 0.15, 0.25, 0.4] # Bins: (0-0.15], (0.15-0.25], (0.25-0.4]
discount_labels = ["Low (0-15%)", "Medium (15-25%)", "High (25%+)"]
supermart["Discount_Bin"] = pd.cut(
    supermart["Discount"], bins=discount_bins, labels=discount_labels, right=True
)

# Calculate average profit and sales per discount bin
profitability_by_discount = (
    supermart.groupby("Discount_Bin")
    .agg(
        Average_Profit=("Profit", "mean"),
        Average_Sales=("Sales", "mean"),
        Total_Sales=("Sales", "sum"),
        Total_Profit=("Profit", "sum"),
        Count=("Order_ID", "count"),
    )
    .reset_index()
)

print("\nProfitability Analysis by Discount Bin:")
profitability_by_discount

```

Profitability Analysis by Discount Bin:

```
Out[ ]:
```

	Discount_Bin	Average_Profit	Average_Sales	Total_Sales	Total_Profit	Count
0	Low (0-15%)	374.703013	1496.062528	3301810	826969.55	2207
1	Medium (15-25%)	373.291945	1501.927450	5837992	1450985.79	3887
2	High (25%+)	376.709195	1491.584615	5817180	1469165.86	3900

```

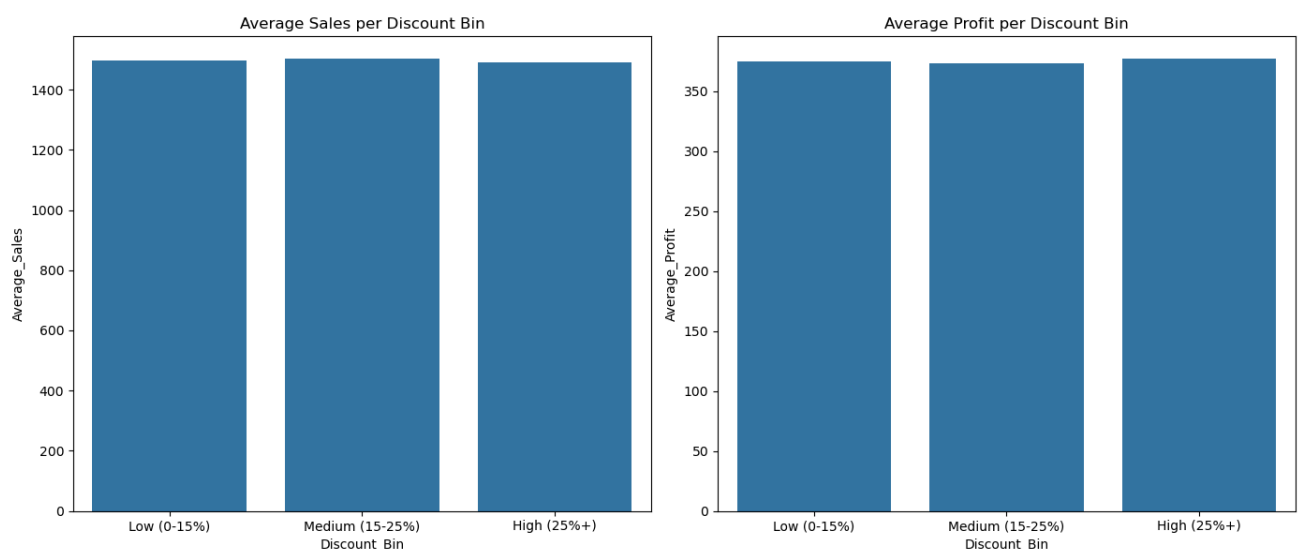
In [ ]: # Visualize average profit/sales by discount bin
plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
sns.barplot(data=profitability_by_discount, x="Discount_Bin", y="Average_Sales")
plt.title("Average Sales per Discount Bin")

plt.subplot(1, 2, 2)
sns.barplot(data=profitability_by_discount, x="Discount_Bin", y="Average_Profit")
plt.title("Average Profit per Discount Bin")

plt.tight_layout()
plt.show()

```



```

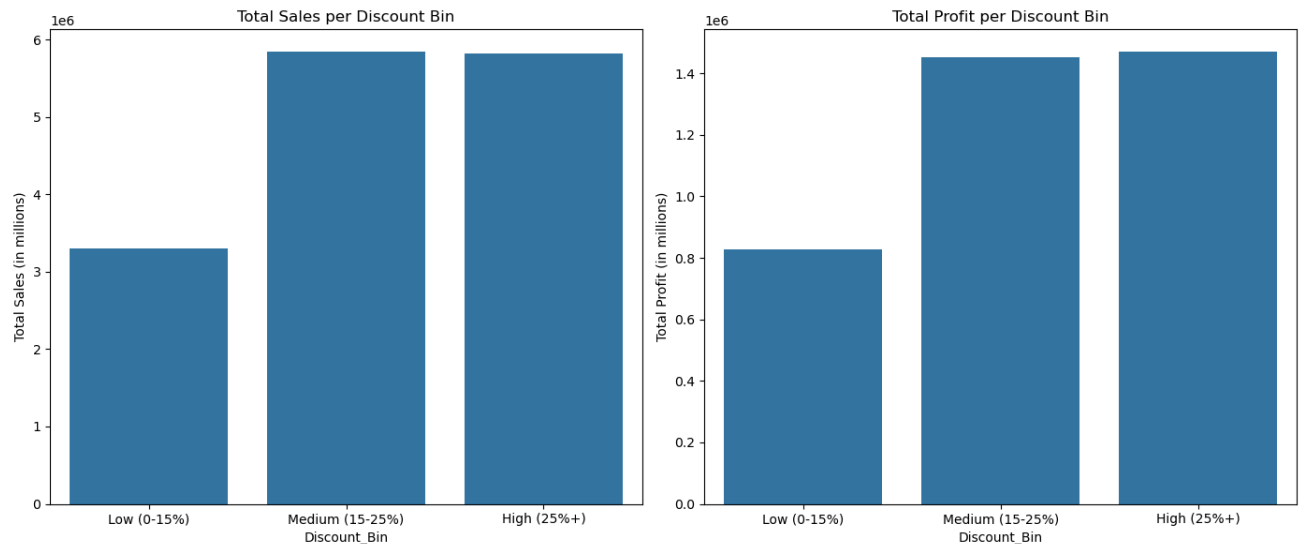
In [39]: #Visualize total profit/sales by discount bin
plt.figure(figsize=(14, 6))

```

```
plt.subplot(1, 2, 1)
sns.barplot(data=profitability_by_discount, x="Discount_Bin", y="Total_Sales")
plt.title("Total Sales per Discount Bin")
plt.ylabel("Total Sales (in millions)") # Adjust if needed

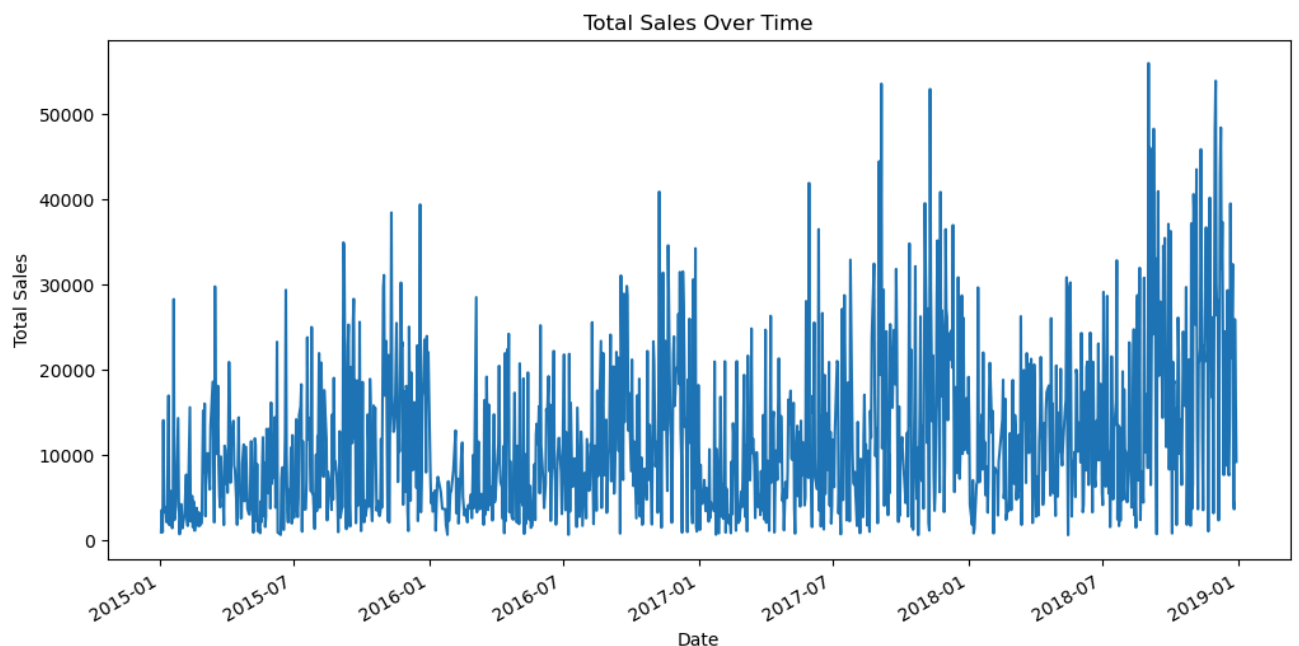
plt.subplot(1, 2, 2)
sns.barplot(data=profitability_by_discount, x="Discount_Bin", y="Total_Profit")
plt.title("Total Profit per Discount Bin")
plt.ylabel("Total Profit (in millions)") # Adjust if needed

plt.tight_layout()
plt.show()
```



In [18]:

```
# 2. Sales Trends Over Time
plt.figure(figsize=(12, 6))
supermart.groupby("Order_Date")["Sales"].sum().plot()
plt.title("Total Sales Over Time")
plt.xlabel("Date")
plt.ylabel("Total Sales")
plt.show()
```

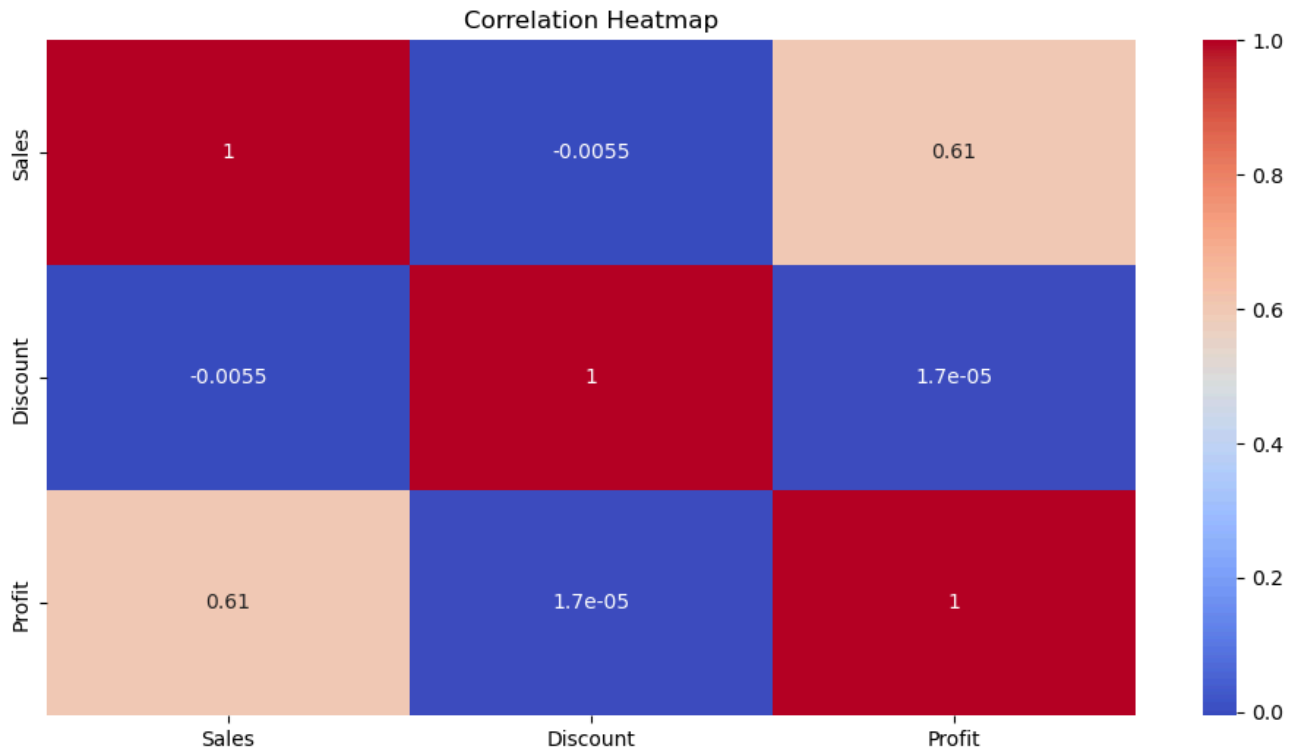


Data Visualization

Visualize the data to identify trends and patterns.

In [19]:

```
# 3. Correlation Heatmap
plt.figure(figsize=(12, 6))
corr_matrix = supermart.select_dtypes(include=["float64", "int64"]).corr()
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

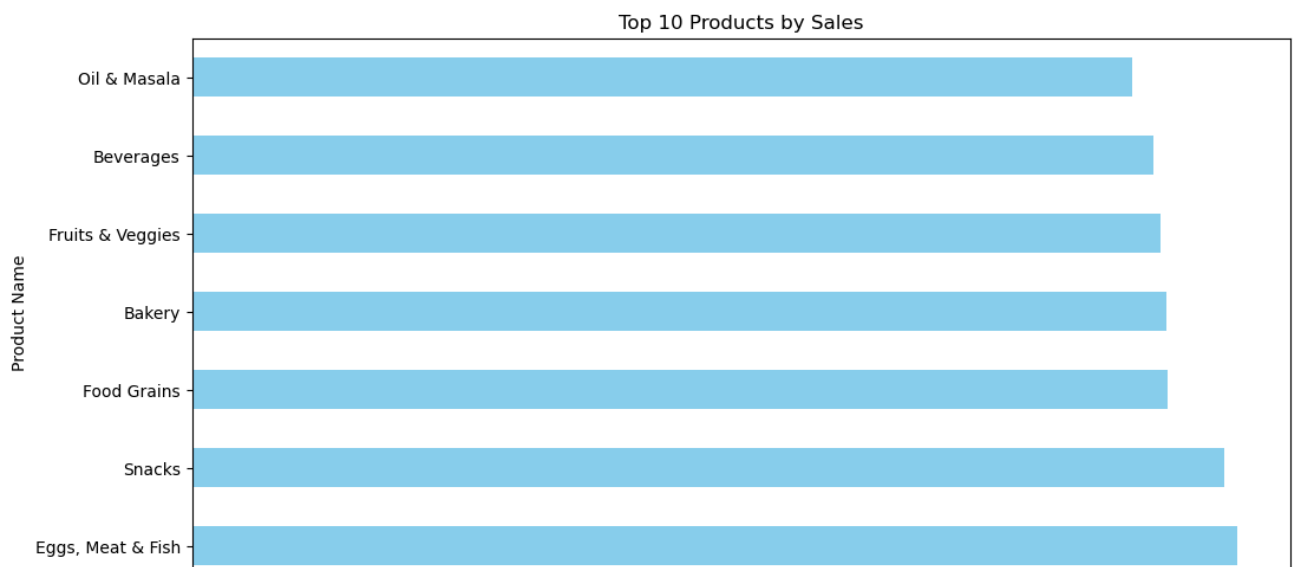


Correlation Analysis show that the most correlated features with Sales are: Profit and Discount

Aggregated Analysis

Group data by categories (e.g., Segment, Category) to analyze sales and profit.

```
In [20]: # Top 10 Products by Sales
top_10_products = supermart.groupby("Category")["Sales"].sum().nlargest(10)
plt.figure(figsize=(12, 6))
top_10_products.plot(kind="barh", color="skyblue")
plt.title("Top 10 Products by Sales")
plt.xlabel("Sales")
plt.ylabel("Product Name")
plt.show()
```



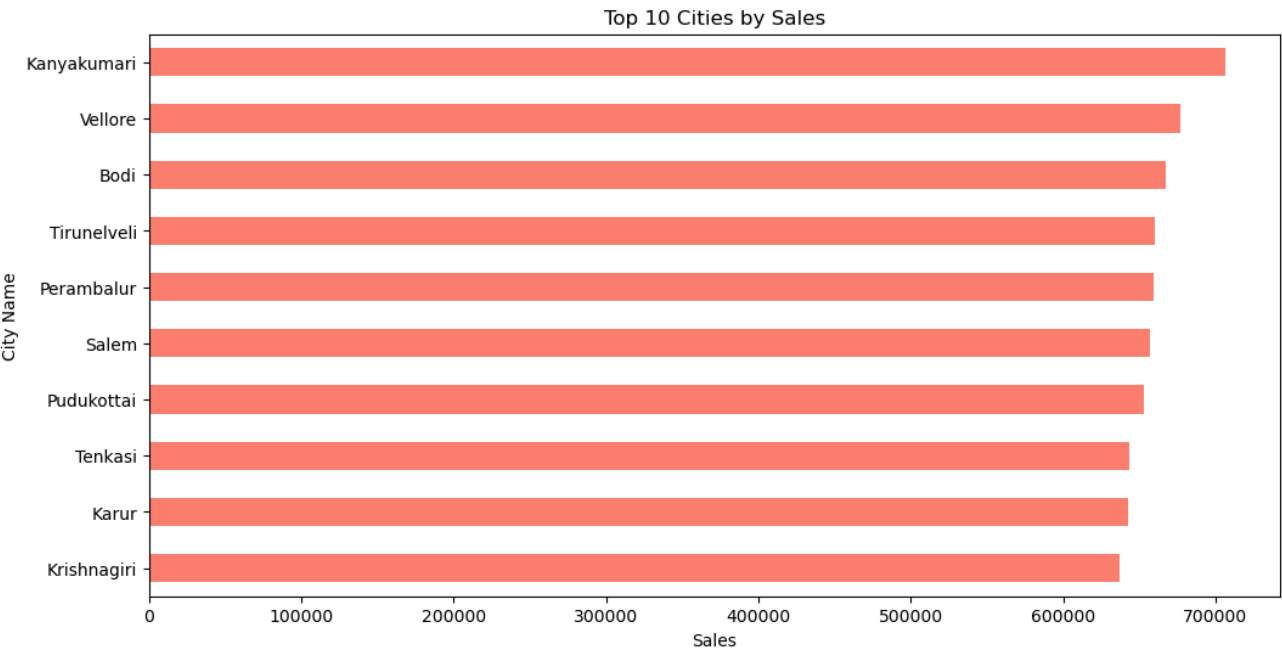
Eggs, Meat, Fish, and Snacks are the top 4 products by sales

Aggregated Analysis

Group data by categories (e.g., Segment, Category) to analyze sales and profit.

```
In [21]: # Top 10 Cities by Sales
top_10_cities = (
    supermart.groupby("City")["Sales"].sum().nlargest(10).sort_values(ascending=True)
)

plt.figure(figsize=(12, 6))
top_10_cities.plot(kind="barh", color="salmon")
plt.title("Top 10 Cities by Sales")
plt.xlabel("Sales")
plt.ylabel("City Name")
plt.show()
```



Kanyakumari Plays a Major Role in Sales

Aggregated Analysis

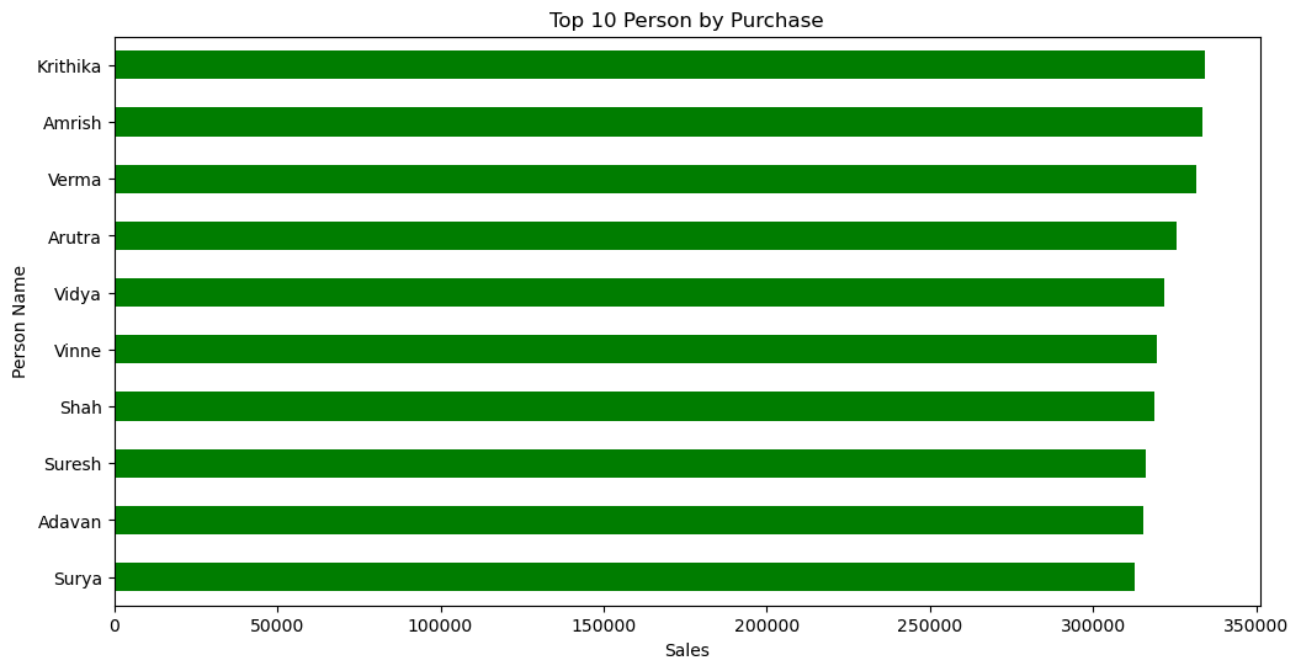
Group data by categories (e.g., Segment, Category) to analyze sales and profit.

```
In [22]: # Top 10 person with Highest purchase
top_10_person = (
    supermart.groupby("Customer_Name")["Sales"]
        .sum()
        .nlargest(10)
        .sort_values(ascending=True)
)

plt.figure(figsize=(12, 6))

top_10_person.plot(kind="barh", color="green")
plt.title("Top 10 Person by Purchase")
plt.xlabel("Sales")
```

```
plt.xlabel("Sales")
plt.ylabel("Person Name")
plt.show()
```



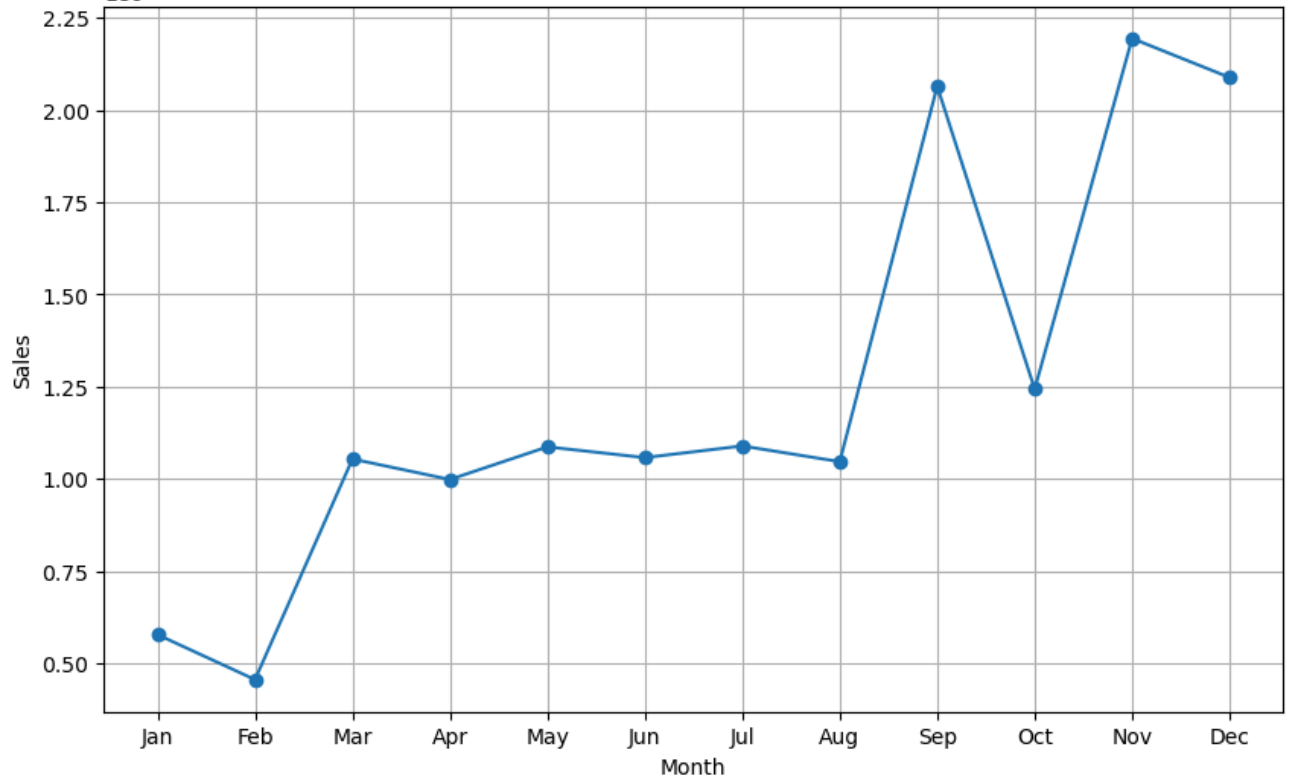
Krithika, Amrish, Verma purchased the most items from the Supermart store.

Aggregated Analysis

Group data by categories (e.g., Segment, Category) to analyze sales and profit.

```
In [23]: # Sum up sales by month
monthly_sales = supermart.groupby("Order_Month")["Sales"].sum().reset_index()
# Sort the data by month
monthly_sales_sorted = monthly_sales.sort_values(by="Order_Month")
# Create the line chart
plt.figure(figsize=(10, 6))
plt.plot(monthly_sales_sorted["Order_Month"], monthly_sales_sorted["Sales"], marker="o")
plt.title("Sales by Month")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.xticks(
    monthly_sales_sorted["Order_Month"],
    [
        "Jan",
        "Feb",
        "Mar",
        "Apr",
        "May",
        "Jun",
        "Jul",
        "Aug",
        "Sep",
        "Oct",
        "Nov",
        "Dec",
    ],
)
plt.grid(True)
plt.show()
```





During the Month of January, and February, the sales are very low.

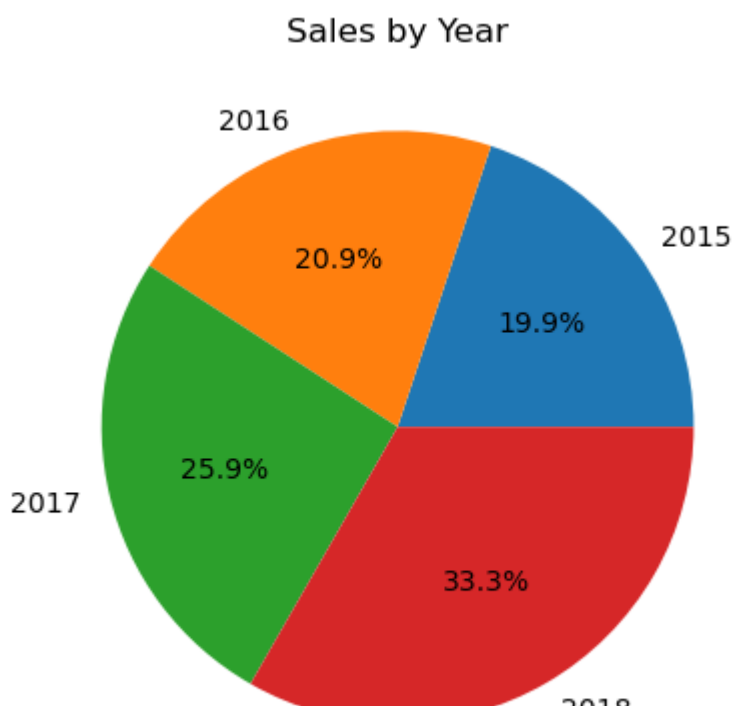
The sales are very high during the month of September, and December.

Aggregated Analysis

Group data by categories (e.g., Segment, Category) to analyze sales and profit.

```
In [ ]: # we group by Year and get the total number of sales for each
Yearly_Sales = supermart.groupby("Order_Year")["Sales"].sum()

# we create a pie chart with the sales by year
plt.pie(Yearly_Sales, labels=Yearly_Sales.index, autopct="%1.1f%%")
plt.title("Sales by Year")
plt.show()
```



The year 2017 and 2018 had more than 50 percent of the total sales which, implies the sales increase as the year increases.

In [40]:

```
import datetime as dt

# Add a small offset (e.g., 1 day) to ensure even the last day's purchases have a recent
snapshot_date = supermart["Order_Date"].max() + dt.timedelta(days=1)
print(f"Snapshot Date for Recency Calculation: {snapshot_date}")

# Aggregate data per customer
rfm_data = supermart.groupby("Customer_Name").agg(
    {
        "Order_Date": lambda x: (
            snapshot_date - x.max()
        ).days, # Recency: Days since last purchase
        "Order_ID": "count", # Frequency: Total number of orders
        "Sales": "sum", # Monetary: Total sales value
    }
)

# Rename columns for clarity
rfm_data.rename(
    columns={"Order_Date": "Recency", "Order_ID": "Frequency", "Sales": "Monetary"},
    inplace=True,
)

# Display the first few rows of the RFM data
print("\nRFM Metrics per Customer:")
rfm_data.head()
```

Snapshot Date for Recency Calculation: 2018-12-31 00:00:00

RFM Metrics per Customer:

Out[40]:

	Recency	Frequency	Monetary
Customer_Name			
Adavan	8	205	315341
Aditi	2	187	299674
Akash	3	196	307488
Alan	7	198	294397
Amrish	4	227	333351

Basic RFM Segmentation (using Quantiles)

In []:

```
# Create labels for R, F, M scores (e.g., 1 is best, 4 is worst for Recency; 4 is best f
r_labels = range(4, 0, -1) # Lower recency (more recent) is better
f_labels = range(1, 5) # Higher frequency is better
m_labels = range(1, 5) # Higher monetary is better

# Assign scores based on quantiles (e.g., 4 quantiles/quartiles)
rfm_data["R_Score"] = pd.qcut(rfm_data["Recency"], q=4, labels=r_labels)
rfm_data["F_Score"] = pd.qcut(rfm_data["Frequency"], q=4, labels=f_labels)
rfm_data["M_Score"] = pd.qcut(rfm_data["Monetary"], q=4, labels=m_labels)
```

```

# Combine scores into a single RFM score string
rfm_data["RFM_Score"] = (
    rfm_data["R_Score"].astype(str)
    + rfm_data["F_Score"].astype(str)
    + rfm_data["M_Score"].astype(str)
)

# Display RFM scores
print("\nRFM Scores per Customer:")
print(rfm_data.head())

# --- Analyze Segments ---
# Example: Count customers in each RFM score segment
print("\nCustomer Counts per RFM Score Segment (Top 10):")
print(rfm_data["RFM_Score"].value_counts().head(10))

# Example: Identify 'Best Customers' (e.g., score 444)
best_customers = rfm_data[rfm_data["RFM_Score"] == "444"]
print(f"\nNumber of Best Customers (Score 444): {len(best_customers)}")
print(best_customers.head())

```

Snapshot Date for Recency Calculation: 2018-12-31 00:00:00

RFM Metrics per Customer:

	Recency	Frequency	Monetary
Customer_Name			
Adavan	8	205	315341
Aditi	2	187	299674
Akash	3	196	307488
Alan	7	198	294397
Amrish	4	227	333351

RFM Scores per Customer:

	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Score
Customer_Name							
Adavan	8	205	315341	1	3	4	134
Aditi	2	187	299674	4	1	3	413
Akash	3	196	307488	3	2	3	323
Alan	7	198	294397	1	2	2	122
Amrish	4	227	333351	2	4	4	244

Customer Counts per RFM Score Segment (Top 10):

RFM_Score

111	4
122	4
244	4
433	3
323	3
344	3
211	3
222	2
334	2
142	2

Name: count, dtype: int64

Number of Best Customers (Score 444): 1

	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Score
Customer_Name							
Surya	2	209	312645	4	4	4	444

Prepare Data for Time Series

In []:

```

# Ensure 'Order_Date' is the index and it's sorted
supermart_ts = supermart.set_index("Order_Date")
supermart_ts.sort_index(inplace=True)

```

```
# Aggregate sales by month
# 'MS' stands for Month Start frequency
monthly_sales_ts = supermart_ts["Sales"].resample("MS").sum()

# Display the first few and last few rows of the monthly sales data
print("Monthly Sales Time Series (First 5):")
print(monthly_sales_ts.head())
print("\nMonthly Sales Time Series (Last 5):")
print(monthly_sales_ts.tail())
```

Monthly Sales Time Series (First 5):

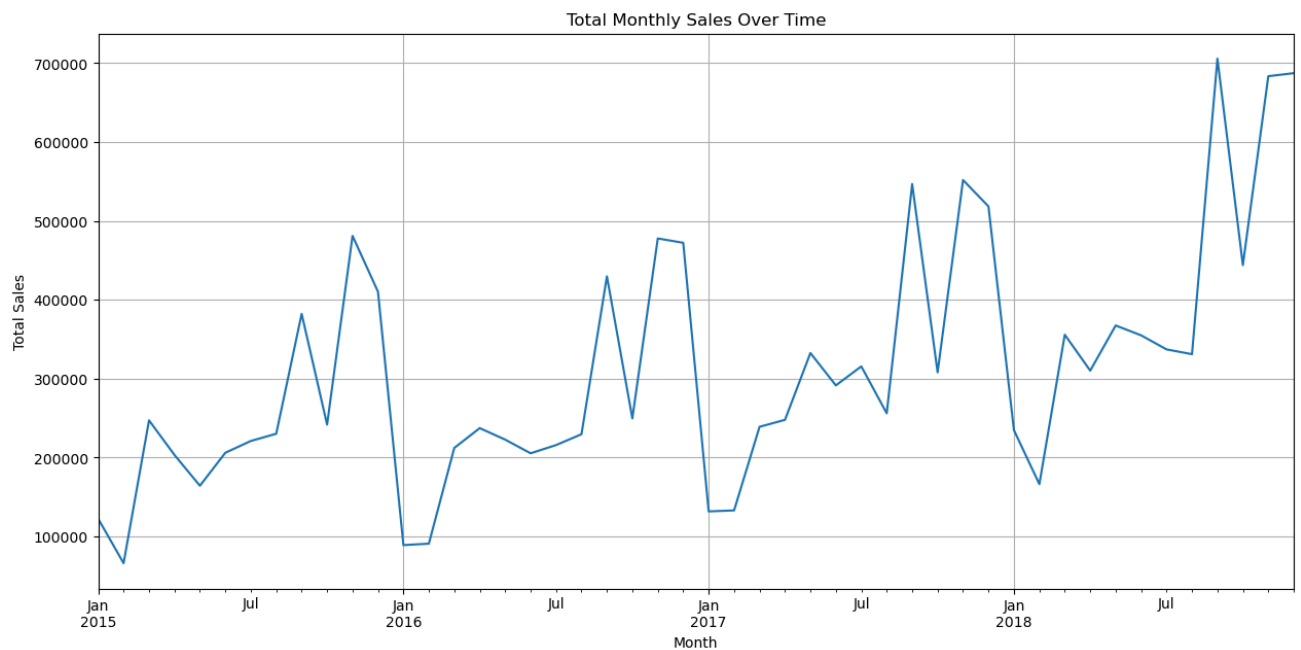
```
Order_Date
2015-01-01    122497
2015-02-01     66030
2015-03-01    247156
2015-04-01    203258
2015-05-01    164263
Freq: MS, Name: Sales, dtype: int64
```

Monthly Sales Time Series (Last 5):

```
Order_Date
2018-08-01    331014
2018-09-01    705680
2018-10-01    443898
2018-11-01    683410
2018-12-01    687245
Freq: MS, Name: Sales, dtype: int64
```

In [44]:

```
# --- Visualize the Monthly Sales ---
plt.figure(figsize=(15, 7))
monthly_sales_ts.plot()
plt.title("Total Monthly Sales Over Time")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.grid(True)
plt.show()
```



Monthly Sales are Increasing Gradually over the year

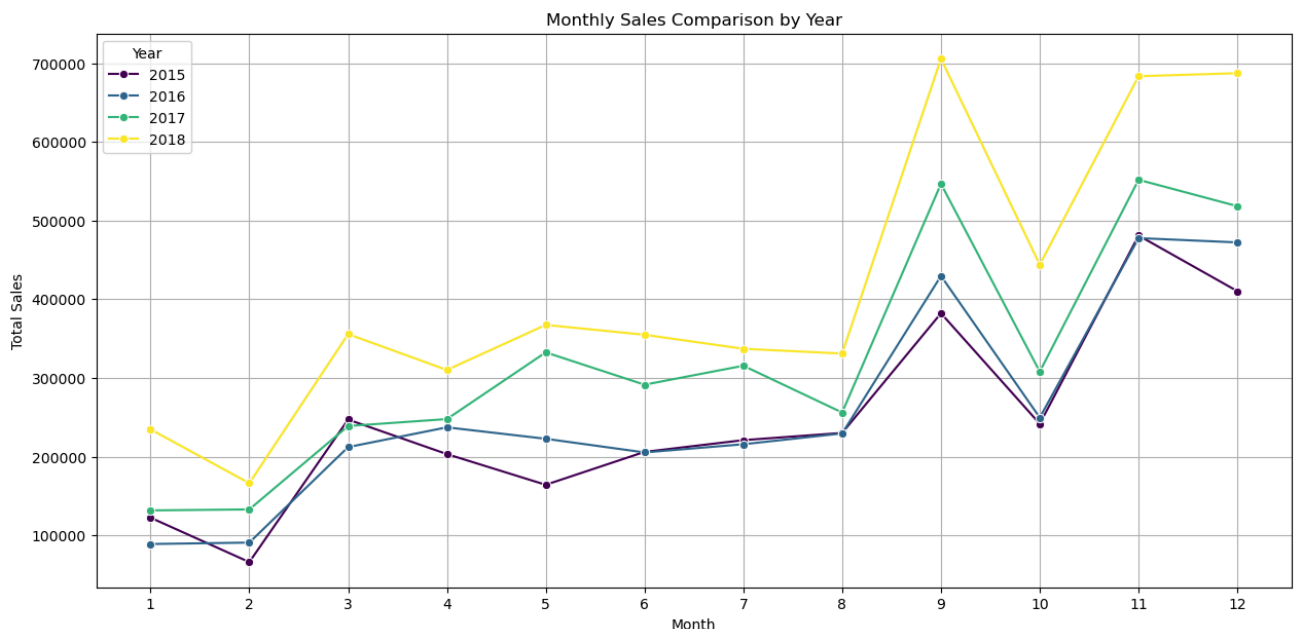
Check for Seasonality (Example: Plotting each year separately)

```

# Create year and month columns for easier plotting if needed, or use pandas time series
monthly_sales_df = monthly_sales_ts.reset_index()
monthly_sales_df["Order_Year"] = monthly_sales_df["Order_Date"].dt.year
monthly_sales_df["Order_Month"] = monthly_sales_df["Order_Date"].dt.month

plt.figure(figsize=(15, 7))
sns.lineplot(
    data=monthly_sales_df,
    x="Order_Month",
    y="Sales",
    hue="Order_Year",
    palette="viridis",
    marker="o",
)
plt.title("Monthly Sales Comparison by Year")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.xticks(range(1, 13))
plt.legend(title="Year")
plt.grid(True)
plt.show()

```



In [29]: `pip install prophet`

Collecting prophet

Downloading prophet-1.1.6-py3-none-win_amd64.whl.metadata (3.6 kB)

Collecting cmdstanpy>=1.0.4 (from prophet)

Downloading cmdstanpy-1.2.5-py3-none-any.whl.metadata (4.0 kB)

Requirement already satisfied: numpy>=1.15.4 in c:\users\tamil\anaconda4\lib\site-packages (from prophet) (1.26.4)

Requirement already satisfied: matplotlib>=2.0.0 in c:\users\tamil\anaconda4\lib\site-packages (from prophet) (3.9.2)

Requirement already satisfied: pandas>=1.0.4 in c:\users\tamil\anaconda4\lib\site-packages (from prophet) (2.2.2)

Collecting holidays<1,>=0.25 (from prophet)

Downloading holidays-0.71-py3-none-any.whl.metadata (34 kB)

Requirement already satisfied: tqdm>=4.36.1 in c:\users\tamil\anaconda4\lib\site-packages (from prophet) (4.66.5)

Collecting importlib-resources (from prophet)

Using cached importlib_resources-6.5.2-py3-none-any.whl.metadata (3.9 kB)

Collecting stanio<2.0.0,>=0.4.0 (from cmdstanpy>=1.0.4->prophet)

Downloading stanio-0.5.1-py3-none-any.whl.metadata (1.6 kB)

Requirement already satisfied: python-dateutil in c:\users\tamil\appdata\roaming\python\python312\site-packages (from holidays<1,>=0.25->prophet) (2.9.0.post0)

Requirement already satisfied: contourpy>=1.0.1 in c:\users\tamil\anaconda4\lib\site-packages (from matplotlib>=2.0.0->prophet) (1.2.0)

```
Requirement already satisfied: cycler>=0.10 in c:\users\tamil\anaconda4\lib\site-packages
(from matplotlib>=2.0.0->prophet) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\tamil\anaconda4\lib\site-pac
kages (from matplotlib>=2.0.0->prophet) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\tamil\anaconda4\lib\site-pac
kages (from matplotlib>=2.0.0->prophet) (1.4.4)
Requirement already satisfied: packaging>=20.0 in c:\users\tamil\appdata\roaming\python\p
ython312\site-packages (from matplotlib>=2.0.0->prophet) (24.1)
Requirement already satisfied: pillow>=8 in c:\users\tamil\anaconda4\lib\site-packages (f
rom matplotlib>=2.0.0->prophet) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\tamil\anaconda4\lib\site-pack
ages (from matplotlib>=2.0.0->prophet) (3.1.2)
Requirement already satisfied: pytz>=2020.1 in c:\users\tamil\anaconda4\lib\site-packages
(from pandas>=1.0.4->prophet) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\tamil\anaconda4\lib\site-packag
es (from pandas>=1.0.4->prophet) (2023.3)
Requirement already satisfied: colorama in c:\users\tamil\appdata\roaming\python\python31
2\site-packages (from tqdm>=4.36.1->prophet) (0.4.6)
Requirement already satisfied: six>=1.5 in c:\users\tamil\appdata\roaming\python\python31
2\site-packages (from python-dateutil->holidays<1,>=0.25->prophet) (1.16.0)
Downloading prophet-1.1.6-py3-none-win_amd64.whl (13.3 MB)
----- 0.0/13.3 MB ? eta --:--:--
----- 0.3/13.3 MB ? eta --:--:--
- ----- 0.5/13.3 MB 1.5 MB/s eta 0:00:09
--- ----- 1.0/13.3 MB 1.6 MB/s eta 0:00:08
--- ----- 1.3/13.3 MB 1.6 MB/s eta 0:00:08
----- 1.6/13.3 MB 1.6 MB/s eta 0:00:08
----- 2.1/13.3 MB 1.7 MB/s eta 0:00:07
----- 2.4/13.3 MB 1.7 MB/s eta 0:00:07
----- 2.9/13.3 MB 1.7 MB/s eta 0:00:07
----- 3.1/13.3 MB 1.7 MB/s eta 0:00:06
----- 3.7/13.3 MB 1.7 MB/s eta 0:00:06
----- 3.9/13.3 MB 1.7 MB/s eta 0:00:06
----- 4.2/13.3 MB 1.7 MB/s eta 0:00:06
----- 4.7/13.3 MB 1.7 MB/s eta 0:00:06
----- 5.0/13.3 MB 1.7 MB/s eta 0:00:05
----- 5.2/13.3 MB 1.7 MB/s eta 0:00:05
----- 5.8/13.3 MB 1.7 MB/s eta 0:00:05
----- 6.0/13.3 MB 1.7 MB/s eta 0:00:05
----- 6.3/13.3 MB 1.7 MB/s eta 0:00:05
----- 6.8/13.3 MB 1.7 MB/s eta 0:00:04
----- 7.1/13.3 MB 1.7 MB/s eta 0:00:04
----- 7.6/13.3 MB 1.7 MB/s eta 0:00:04
----- 7.9/13.3 MB 1.7 MB/s eta 0:00:04
----- 8.4/13.3 MB 1.7 MB/s eta 0:00:03
----- 8.7/13.3 MB 1.7 MB/s eta 0:00:03
----- 8.9/13.3 MB 1.7 MB/s eta 0:00:03
----- 9.2/13.3 MB 1.7 MB/s eta 0:00:03
----- 9.2/13.3 MB 1.7 MB/s eta 0:00:03
----- 9.4/13.3 MB 1.6 MB/s eta 0:00:03
----- 9.4/13.3 MB 1.6 MB/s eta 0:00:03
----- 9.4/13.3 MB 1.6 MB/s eta 0:00:03
----- 9.4/13.3 MB 1.6 MB/s eta 0:00:03
----- 9.4/13.3 MB 1.6 MB/s eta 0:00:03
----- 9.7/13.3 MB 1.4 MB/s eta 0:00:03
----- 10.0/13.3 MB 1.4 MB/s eta 0:00:03
----- 10.0/13.3 MB 1.4 MB/s eta 0:00:03
----- 10.0/13.3 MB 1.4 MB/s eta 0:00:03
----- 10.2/13.3 MB 1.3 MB/s eta 0:00:03
----- 10.7/13.3 MB 1.3 MB/s eta 0:00:02
----- 10.7/13.3 MB 1.3 MB/s eta 0:00:02
----- 11.3/13.3 MB 1.3 MB/s eta 0:00:02
----- 11.5/13.3 MB 1.3 MB/s eta 0:00:02
----- 11.8/13.3 MB 1.3 MB/s eta 0:00:02
----- 12.1/13.3 MB 1.3 MB/s eta 0:00:01
----- 12.6/13.3 MB 1.3 MB/s eta 0:00:01
----- 12.8/13.3 MB 1.3 MB/s eta 0:00:01
----- 13.1/13.3 MB 1.3 MB/s eta 0:00:01
----- 13.3/13.3 MB 1.3 MB/s eta 0:00:00
```

```

Downloading cmdstanpy-1.2.5-py3-none-any.whl (94 kB)
Downloading holidays-0.71-py3-none-any.whl (917 kB)
----- 0.0/917.9 kB ? eta -:--:--
----- 262.1/917.9 kB ? eta -:--:--
----- 524.3/917.9 kB 1.5 MB/s eta 0:00:01
----- 917.9/917.9 kB 1.6 MB/s eta 0:00:00
Using cached importlib_resources-6.5.2-py3-none-any.whl (37 kB)
Downloading stanio-0.5.1-py3-none-any.whl (8.1 kB)
Installing collected packages: stanio, importlib-resources, holidays, cmdstanpy, prophet
Successfully installed cmdstanpy-1.2.5 holidays-0.71 importlib-resources-6.5.2 prophet-1.
1.6 stanio-0.5.1
Note: you may need to restart the kernel to use updated packages.

```

Displaying the Month sales of the data

```

In [ ]: # --- Import Prophet ---
        from prophet import Prophet

        # --- Prepare Data for Prophet ---
        # Prophet requires columns named 'ds' (for date) and 'y' (for the value to forecast)
        monthly_sales_prophet = monthly_sales_ts.reset_index()
        monthly_sales_prophet.rename(columns={"Order_Date": "ds", "Sales": "y"}, inplace=True)

        print("Data prepared for Prophet (first 5 rows):")
        monthly_sales_prophet.head()

```

Data prepared for Prophet (first 5 rows):

```

Out[ ]:

```

	ds	y
0	2015-01-01	122497
1	2015-02-01	66030
2	2015-03-01	247156
3	2015-04-01	203258
4	2015-05-01	164263

Prophet Sales Analysis for next 12 months

```

In [47]: # --- Instantiate and Fit the Model ---
        # We'll use default settings for now
        model = Prophet()
        model.fit(monthly_sales_prophet)

        # --- Create Future Dataframe ---
        # Let's forecast for the next 12 months
        future = model.make_future_dataframe(periods=12, freq="MS") # MS for Month Start
        print("\nFuture dataframe created (last 5 rows):")
        print(future.tail())

        # --- Make Predictions ---
        forecast = model.predict(future)

        # Display some forecast columns (ds, yhat, yhat_lower, yhat_upper)
        print("\nForecast results (last 5 rows):")
        print(forecast[["ds", "yhat", "yhat_lower", "yhat_upper"]].tail())

```

```

11:42:27 - cmdstanpy - INFO - Chain [1] start processing
11:42:27 - cmdstanpy - INFO - Chain [1] done processing
Future dataframe created (last 5 rows):

```

```

ds
55 2019-08-01
56 2019-09-01
57 2019-10-01
58 2019-11-01
59 2019-12-01

```

Forecast results (last 5 rows):

	ds	yhat	yhat_lower	yhat_upper
55	2019-08-01	469445.235443	435295.780897	503676.395889
56	2019-09-01	715385.696709	682352.934390	747532.178545
57	2019-10-01	525179.433099	493288.962247	558745.865230
58	2019-11-01	769777.583141	737080.363249	804886.104226
59	2019-12-01	723754.856525	688277.478938	757316.076727

Forecast plot for the next 12 months

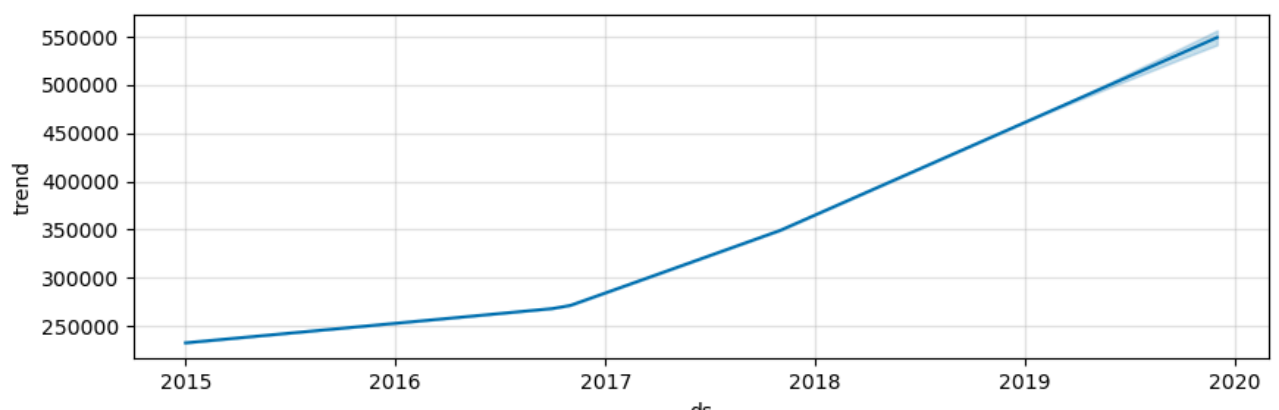
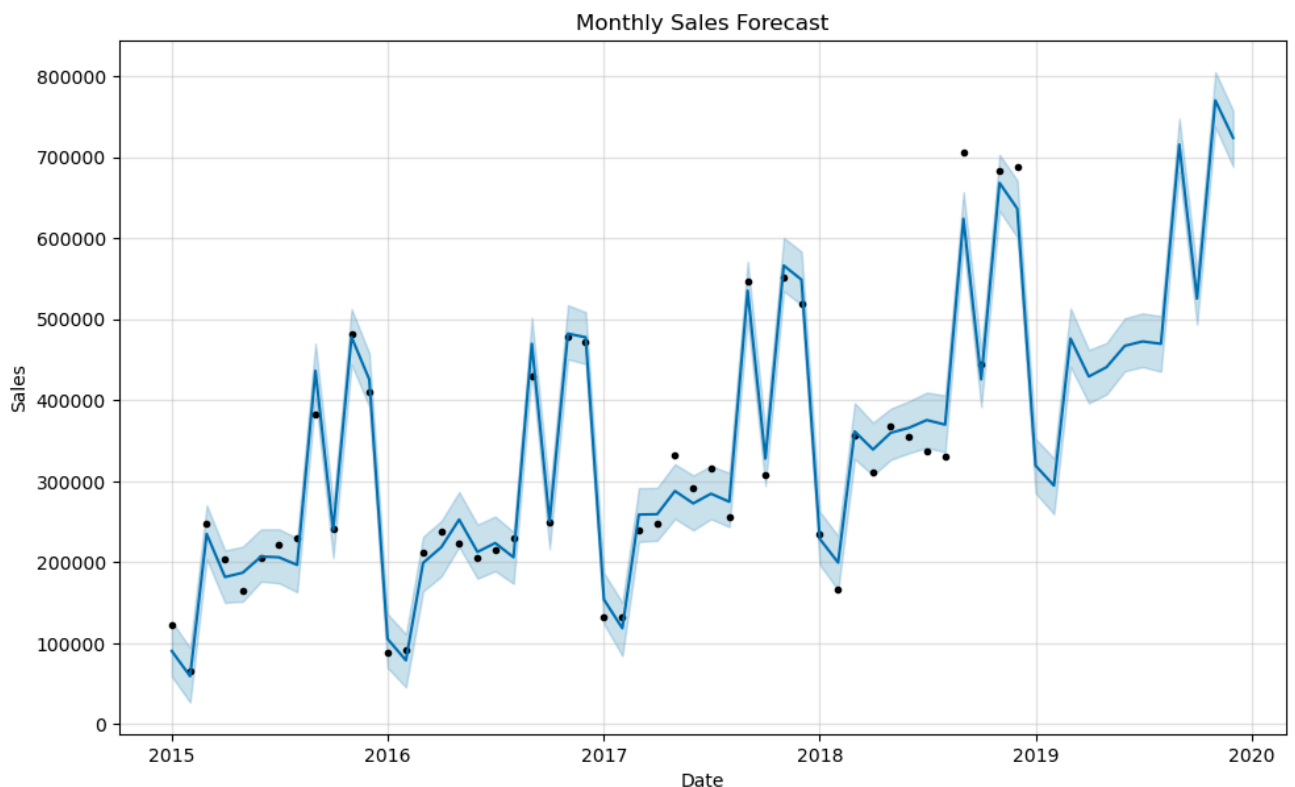
In [48]:

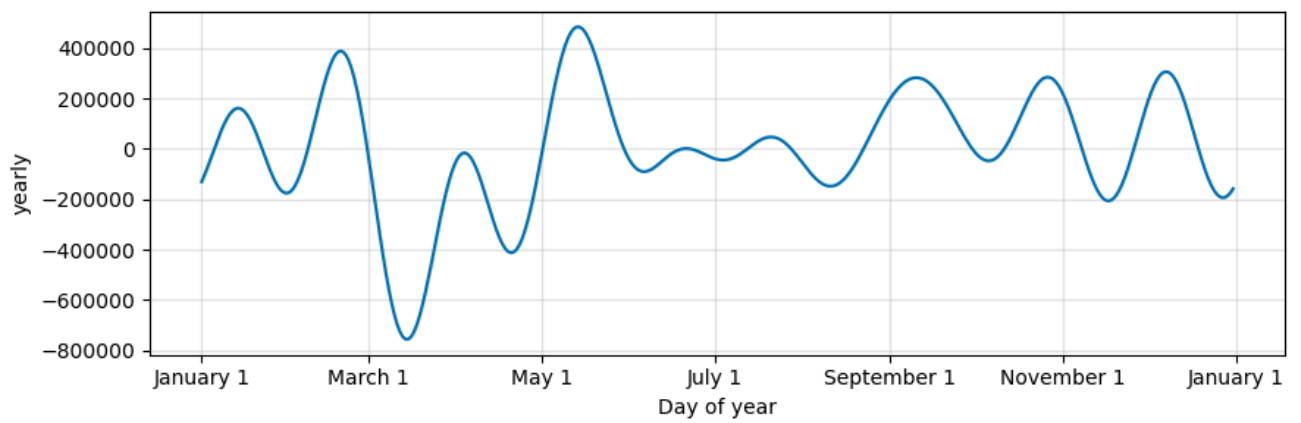
```

# --- Plot the Forecast ---
fig1 = model.plot(forecast)
plt.title("Monthly Sales Forecast")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.show()

# --- Plot Forecast Components ---
# This shows the trend, yearly seasonality, etc.
fig2 = model.plot_components(forecast)
plt.show()

```





Conclusion

The analysis provided insights into sales trends, customer behavior, and the impact of discounts on sales and profit.

The RFM analysis helped identify customer segments, and the Prophet model forecasted future sales trends.

This information can be used for targeted marketing, inventory management, and strategic planning.